

Design Your Personal Agent: Engineering Spec (Code: FL-06)

Track:	General AI Fluency	Phase:	Build (Core)
Type:	Assignment Deliverable	Workload:	4 Hours

1. JOB TO BE DONE, TARGET USER & ESTIMATED BUILD SCOPE

Agent Identity: Automated Technical Research Scout & Vulnerability Digest Agent.

Job to Be Done: Monitor technical feeds, CVE security advisories, and framework release notes, filter out marketing noise, summarize breaking architectural changes, and draft actionable weekly engineering briefs.

Target User & Frequency: Myself (Full-Stack / Systems Developer). Runs automatically every weekday morning at 8:00 AM.

Estimated Build Scope: Roughly 8 to 10 build hours. Focused strictly on ingestion, synthesis, validation, and markdown delivery.

2. REQUIRED TOOLS, DATA SOURCES & REALISTIC ACCESS PLAN

Tool / Data Source	Functional Role in Agent Loop	Access & Authentication Plan
GitHub Releases & Commits	Scrapes release changelogs for core frameworks (FastAPI, React, PyTorch).	GitHub REST API using free personal access token with read-only public repo scope.
NVD / CVE Feeds	Pulls live security vulnerability advisories relevant to the active tech stack.	NIST NVD Public REST API endpoint (free tier, zero authentication required).
Local Knowledge Base	Read-only access to my current active projects and library dependency lists.	Local MCP file system connector reading local `requirements.txt` and `package.json`.
Digest Delivery Channel	Outputs the formatted markdown brief to local folder or webhook note.	Local MCP `write_file` tool to save daily summary into `./digests/YYYY-MM-DD.md`.

3. DRAFT SYSTEM INSTRUCTIONS & OPERATING RULES

You are the Technical Research Scout Agent. Your goal is to keep the developer informed of critical framework updates and security alerts without causing information overload. Operating Rules: 1. Ingestion: Scan connected sources for breaking releases and security patches published within the last 24 hours. 2. Filtering: Match findings against the local dependency list. Ignore cosmetic marketing updates or unverified third-party rumors. 3. Synthesis: For each relevant finding, write: 1. Component Name & Version, 2. Core Breaking Change or Risk, 3. Required Developer Action. 4. Tone & Style: Direct, technical, and objective. Use plain English and no corporate buzzwords. Never use Em dashes.

4. FIVE EVALUATION CASES (TEST BENCH BEFORE BUILDING)

Eval Case	Input Scenario	Expected Agent Behavior & Success Criteria
Eval 1: Critical Patch	FastAPI releases security patch fixing query parameter injection.	Agent flags it as Priority 1 (Action Required), references local usage in project, and provides exact upgrade command.
Eval 2: Irrelevant Noise	Unrelated marketing post from a cloud vendor about AI synergy.	Agent discards the item completely during Stage 1 filtering with zero mention in the final brief.
Eval 3: Breaking Change	Node.js updates LTS deprecating an older TLS protocol method.	Agent lists breaking change, checks local codebase for affected method calls, and notes impacted files.
Eval 4: Empty Feed	Zero relevant updates or security notices published in the past 24 hours.	Agent outputs a clean "All systems stable, no critical updates detected" note without inventing synthetic entries.
Eval 5: Conflicting Info	Community post claims zero-day bug, but no CVE or official patch exists.	Agent marks status as "Unconfirmed / Under Watch" and warns the developer before recommending changes.

5. RISKS & OPERATIONAL GUARDRAILS

Hard Safety Rules (What the Agent Must NEVER Do):

- Must NEVER execute automated package updates or modify `package.json` / `requirements.txt` independently.
- Must NEVER delete existing digest archives or alter source code files outside designated output directories.
- Must NEVER send outbound emails or public social notifications without explicit manual confirmation.

Mandatory Human-in-the-Loop Triggers (When Agent Must Request Confirmation):

1. Flagging a dependency upgrade that introduces major breaking syntax changes.
2. Any CVE scored above 8.5 (Critical) that requires immediate manual server patching.

6. PLATFORM CHOICE JUSTIFICATION

Chosen Platform: Claude Project paired with Local MCP Connectors (Python / Node).

Why This Platform: Provides free local execution, allows complete control over file system reads/writes, supports custom system instructions, and eliminates reliance on expensive paid cloud agent platforms.

Justified Against Alternative (Custom GPT / Paid Cloud Agent): Custom GPTs run inside closed OpenAI sandboxes and cannot access private local project dependencies or write local markdown files without paid API bridges. A local MCP setup is 100% free and private.